Date _____ Expt. No. _____

Expt. Name _____ Page No. 49.

Aim:- To implement an AVL tree in C with insertion operation & perform necessary rotation (LL, RR, LR, RL) to maintain height - balanced property of the tree.

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *left * right
    int height;
};

int height ( struct node *n) {
        if (n == NULL)
            return 0;
        return n -> height;
    }
int max(int a, int b) {
    return (a > b) ? a : b;
    }
struct node* createNode (int key) {
    struct node* newNode =(struct node*) malloc
            (sizeof (structnode));
    newNode -> data = key;
    newNode -> left = newNode -> right = NULL;
    newNode -> height = 1;
    return newNode;
    }
```

Teacher's Signature: _____

Date _____ Expt. No. _____

Expt. Name _____ Page No. 50.

```
// Right Rotate (LL Rotation)
Struct node* rightRotate (Struct node* y) {
    Struct node* x = y -> left;
    struct node* T2 = x -> right;

x -> right = y;
y -> left = T2;
y -> height = max (height (y->left), height (y->right)) +1;
x -> height = max (height (x -> left), height (x->right)) +1;
return x;
}
struct node* leftRotate (struct node* x) {
    struct node *y = x -> right;
    struct node *T2 = y -> left;
     y -> left = x;
     x -> right = T2;
x -> height = max (height (x->left), height (x->right))
y -> height = max (height (y -> left), height (y->right)) +1; +1;
return y;
}
int getBalance (struct node *n) {
    if (n == NULL)
     return 0;
     return height (n->left) - height (n->right);
}
Struct node* int insert (Struct node* node, int key) {
    if (node == NULL)
     return createNode (key);
if (key < node -> data)
    node -> left = insert (node -> left, key);
```

Teacher's Signature: _____

```
else if (key > node->data)
    node->right = insert(node->right, key);
else
    return node;
node->height = 1 + max(height(node->left), height(node->right));
int balance = getBalance(node);
if (balance > 1 && key < node->left->data)
    return rightRotate(node);
if (balance < -1 && key > node->right->data)
    return leftRotate(node);

if (balance > 1 && key > node->left->data) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
if (balance < -1 && key < node->right->data)
    {
    node->right = rightRotate(node->right);
    return leftRotate(node);
    }
return node;
}
void preorder(struct node *root) {
    if (root != NULL) {
    printf("%d", root->data);
    preorder(root->left);
    preorder(root->right);
    }
}
```

Teacher's Signature: _____

Output

How many values to insert ? 7
Enter values:
30 20 40 10 25 35 50

Preorder Traversal of AVL Tree:
30 20 10 25 40 35 50

output

Enter number of vertices: 3
Enter adjacency matrix (3×3):
0 1 0
1 0 1
0 1 0

Adjacency Matrix
0 1 0
1 0 1
0 1 0

Date _____ Expt. No. _____
Expt. Name _____ Page No. 52.

```
int main() {
    struct node *root = NULL;
    int n, x;
    printf("How many values to insert? ");
    scanf("%d", &n);
    printf("Enter values:\n");
    for(int i=0; i<n; i++) {
        scanf("%d", &x);
        root = insert(root, x);
    }
    printf("\nPreorder Traversal of AVL Tree:");
    preorder(root);
    return 0;
}
```

Program 2 To implement graph representation using adjacency matrix in C.

```
#include<stdio.h>
int main() {
    int n, i, j;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int graph[n][n];
    printf("Enter adjacency matrix (%d x %d):\n", n, n);
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d", &graph[i][j]);
}
```

Teacher's Signature: _____

Output

Enter number of vertices : 4

Enter the adjacency matrix (4 x 4):

0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0

Enter starting vertex for DFS : 0
Enter starting vertex for BFS : 0

So DFS Traversal : 0 1 3 2
BFS Traversal : 0 1 2 3

```
printf("\n Adjacency Matrix :\n");
for(i=0 ; i<n; i++){
for(j=0 ; j<n ; j++)
{
printf("%d", graph[i][j]);
}
printf("\n");
```

Program 3 To implement graph traversal techniques using Depth first search (DFS) & Breadth first search (BFS) using an adjacency matrix representation.

```
#include<stdio.h>
#define MAX 100
int graph[MAX][MAX], visited[MAX];
int queue[MAX], front = -1, rear = -1;
void DFS(int vertex, int n){
printf("%d", vertex);
visited[vertex] = 1;
for(int i=0; i<n; ++i){
if(graph[vertex][i] == 1 && !visited[i])
{
DFS(i, n);
}
}
}
```

Teacher's Signature

```
void BFS(int start, int n){
    front = rear = -1;
queue[++rear] = start;
    visited[start] = 1;
    while (front != rear){
        int node = queue[++front];
        printf("%d", node);

    for(int i = 0; i<n; i++){
        if(graph[node][i] == 1 && !visited[i]){
    queue[++rear]=i;
        visited[i] = 1;
        }
    }
}
}
    int main()
        int n, i, j, start;
    printf("Enter number of vertices: ");
        scanf("%d", &n);
    printf("Enter adjacency matrix (%dx%d):\n",
        n, n);
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d", &graph[i][j]);
```

Teacher's Signature:

```
for (i=0; i<n ; i++) visited[i] = 0;
printf("\n Enter starting vertex for DFS:");
scanf("%d", &start);
printf(" DFS Traversal :");
DFS (start, n);

for (i=0; ; i<n ; i++) visited[i] = 0;
printf("\n\n Enter starting vertex for BFS:");
scanf("%d", &start);
printf(" BFS Traversal: ");
 BFS ( start, n);
 return 0;
}
}
 return 0;
}
```

Teacher's Signature: